

Document sur les Outils de la Qualité du Code **en Python**

Un guide complet pour des projets Python robustes et maintenables

Fourni par : Mlle Metra Phirielle VANDJY BOUNDZANGA

Etudiante en Master II Programmation à GROUPE SUP'INFO

metraphirielle@gmail.com

Module : Qualité Logicielle / Assurance Qualité

Table des matières

Introduction	4
1.1 pytest.....	5
2. Outils d'Analyse Statique	5
2.1 Pylint.....	5
2.2 flake8.....	6
2.3 SonarQube (avec plugin Python)	6
3. Outils de Sécurité.....	7
3.1 Dependabot	7
3.2 Bandit.....	7
4. Outils de Documentation	7
4.1 Swagger (via drf-yasg pour Django).....	7
4.2 Sphinx.....	8
5. Outils d'Analyse de Performance	8
5.1 cProfile.....	8
5.2 memory_profiler	9
5.3 py-spy.....	9
6. Bonnes Pratiques Complémentaires	9
7. Conclusion.....	10
8. Cas d'utilisation de chaque outils et pipeline	11
8.1. pytest.....	11
8.2. Pylint.....	12
8.3. flake8.....	13
8.4. SonarQube	14
8.5. Dependabot	15
8.6. Bandit.....	15
8.7. Swagger (drf-yasg).....	16
8.8. Sphinx	16
8.9. cProfile.....	17
8.10. memory_profiler	17
8.11. py-spy.....	18

8.12. Pipeline CI/CD Exemple :19

Impact sur la Qualité.....19

Introduction

La qualité du code est un pilier essentiel du développement logiciel, garantissant la fiabilité, la maintenabilité et la performance des applications. Dans l'écosystème Python, riche en bibliothèques et frameworks, il est crucial d'adopter les bons outils pour analyser, tester et optimiser son code.

Ce document présente **une sélection d'outils incontournables** dédiés à l'amélioration de la qualité du code Python. Que vous soyez débutant ou développeur expérimenté, vous y trouverez des solutions pour :

- **Détecter les bugs** et les vulnérabilités (**pytest, Pylint**),
- **Automatiser les tests** et la revue de code (**SonarQube, flake8**),
- **Sécuriser les dépendances** (**Dependabot, Bandit**),
- **Documenter efficacement** (**Swagger, Sphinx**),
- **Optimiser les performances** (**cProfile, py-spy**).

Chaque outil est décrit avec ses **fonctionnalités clés** et un **lien officiel** pour faciliter son adoption. En les intégrant à votre workflow, vous pourrez produire un code plus robuste, efficace et facile à maintenir, tout en respectant les standards de la communauté Python.

1. Outils de Test

1.1 pytest

1.1 pytest

🔗 **Site officiel** : <https://docs.pytest.org/>

🔗 **Description** : pytest est un framework de test puissant et flexible pour Python. Il permet d'écrire des tests unitaires, fonctionnels et d'intégration de manière simple et concise.

🔗 **Fonctionnalités clés** :

- Syntaxe intuitive pour écrire des tests (la détection des bugs).
- Supporte les tests paramétrés et une prise en charge des **fixtures** pour une réutilisation du code.
- Permet une **couverture de code** élevée avec des plugins comme `pytest-cov`.
- Tests paramétrés pour éviter la duplication.
- Génère des rapports de test détaillés, incluant les échecs, les tests ignorés et les temps d'exécution.

2. Outils d'Analyse Statique

2.1 Pylint

🔗 **Site officiel** : <https://pylint.org/>

🔗 **Description** : Pylint est un outil d'analyse statique qui vérifie le code Python pour détecter les erreurs, les problèmes de style (PEP 8) et les violations de conventions, les problèmes de conception, les problèmes de conception.

Fonctionnalités clés :

- Vérification de la **complexité cyclomatique**.
- Détection des variables inutilisées ou des fonctions trop longues.
- Notation du code pour évaluer sa qualité.
- Assure la conformité aux standards PEP 8.
- Améliore la maintenabilité en encourageant un code propre et bien structuré.

2.2 flake8

 **Site officiel :** <https://flake8.pycqa.org/>

 **Description :** Combinaison de `pycodestyle` (PEP 8), `pyflakes` (erreurs syntaxiques) et `mccabe` (complexité).

Fonctionnalités clés :

- Rapide et léger, idéal pour l'intégration dans les pipelines CI/CD.

2.3 SonarQube (avec plugin Python)

 **Site officiel :** <https://www.sonarsource.com/>

 **Description :** SonarQube est une plateforme open source pour l'inspection continue de la qualité du code. Elle prend en charge Python via des plugins.

Fonctionnalités clés :

- Détection de **bugs**, **vulnérabilités**, et **duplications de code**.
- Métriques de **dette technique** et **maintenabilité**.
- Fournit des rapports détaillés pour suivre l'évolution de la qualité.

3. Outils de Sécurité

3.1 Dependabot

 **Site officiel** : <https://dependabot.com/>

 **Description** : Dependabot est un outil qui surveille les dépendances d'un projet et propose automatiquement des mises à jour pour corriger les vulnérabilités.

 **Fonctionnalités clés** :

- Intégration native avec **GitHub**.
- Alertes (les risques liés aux failles de sécurité connues) pour les **CVE** (Common Vulnerabilities and Exposures).
- Garantit que les bibliothèques utilisées sont à jour et sécurisées.

3.2 Bandit

 **Site officiel** : <https://bandit.readthedocs.io/>

 **Description** : Scanne le code Python pour détecter les vulnérabilités de sécurité.

 **Fonctionnalités clés** :

- Détection des **injections SQL**, **hardcoded passwords**, etc.

4. Outils de Documentation

4.1 Swagger (via drf-yasg pour Django)

 **Site officiel** : <https://swagger.io/>

✚ **Description** : Swagger est un outil de documentation interactive pour les API REST. En Python, il peut être utilisé avec des bibliothèques comme drf-yasg pour Django REST Framework.

✚ **Fonctionnalités clés** :

- Génère une documentation interactive pour les API.
- Facilite la compréhension et l'utilisation des endpoints par les développeurs et les utilisateurs finaux (Interface visuelle).

4.2 Sphinx

✚ **Site officiel** : <https://www.sphinx-doc.org/>

✚ **Description** : Outil de documentation utilisé par Python officiellement (ex : docs.python.org).

✚ **Fonctionnalités clés** :

- Supporte le format **reStructuredText** et **Markdown**.

5. Outils d'Analyse de Performance

5.1 cProfile

✚ **Documentation officielle** : <https://docs.python.org/3/library/profile.html>

✚ **Description** : cProfile est un module intégré à Python pour profiler l'exécution du code.

✚ **Fonctionnalités clés** :

- Identifie les **fonctions gourmandes en temps CPU**.

5.2 memory_profiler

- ✚ **Site officiel** : <https://pypi.org/project/memory-profiler/>
- ✚ **Description** : **memory_profiler** mesure l'utilisation mémoire ligne par ligne.
- ✚ **Fonctionnalités clés** :
 - Détecte les **fuites de mémoire**.

5.3 py-spy

- ✚ **Site officiel** : <https://github.com/benfred/py-spy>
- ✚ **Description** : py-spy est un profiler non intrusif pour les applications en production.
- ✚ **Fonctionnalités clés** :
 - Aucune modification du code nécessaire.

6. Bonnes Pratiques Complémentaires

- ❖ **Formateur de code** : **Black** (<https://black.readthedocs.io/>) pour un style cohérent.
- ✓ **Vérification de types** : **mypy** (<https://mypy.readthedocs.io/>).

7. Conclusion

Ces outils couvrent tous les aspects de la qualité logicielle en Python :

Tests, Analyse statique, Sécurité, Documentation et Performance.

Intégrez-les à votre workflow pour des projets **robustes, sécurisés et faciles à maintenir.**

8. Cas d'utilisation de chaque outils et pipeline

8.1. pytest

- **Utilisation :**
 - Installer : `pip install pytest pytest-cov`.
 - Écrire des tests dans `test_*.py`.
 - Exécuter : `pytest --cov=./ --cov-report=html`.

- **Exemple sur l'éditeur:**

python

```
# test_exemple.py
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
```

- **Utiliser des fixtures :**

python

```
import pytest

@pytest.fixture
def donnees():
    return {"cle": "valeur"}

def test_donnees(donnees):
    assert donnees["cle"] == "valeur"
```

- **Intégration CI/CD (GitHub Actions) dans un fichier yaml :**

yaml

```
name: Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Configurer Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.9'
      - name: Installer dépendances
        run: pip install pytest pytest-cov
      - name: Exécuter tests
        run: pytest --cov=./ --cov-report=xml
```

- **Impact :**
 - Réduit les bugs (ex. : 95 % des régressions détectées avec 80 % de couverture).
 - Améliore la fiabilité.

8.2. Pylint

- **Utilisation :**
 - Installer : `pip install pylint`.
 - Exécuter : `pylint fichier.py`.
- **Exemple :**

python

```
# mauvais_code.py
def fonction(a,b):
    x=a+b
    return x
```

❑ **Résultat :** Signale des espaces manquants, noms de variables courts.

❑ **Corrigé :**

python

```
# bon_code.py
def fonction(a, b):
    resultat = a + b
    return resultat
```

- **Intégration CI/CD :**

yaml

```
- name: Exécuter Pylint
  run: pylint *.py --fail-under=8
```

- **Impact :**

- Conforme à PEP 8, améliore la lisibilité.
- Exemple : Score Pylint >8 réduit le temps de refactoring de 30 %.

8.3. flake8

- **Utilisation :**

- Installer : pip install flake8.
- Exécuter : flake8 fichier.py.

- **Exemple : Détecte imports inutilisés, violations PEP 8**

```
python

import os  # inutilisé
def fonction():
    x = 1  # trop d'espaces
```

- **Intégration CI/CD :**

```
yaml

- name: Exécuter flake8
  run: flake8 . --max-line-length=88
```

- **Impact :**
 - Améliore la cohérence du code.
 - Exemple : Réduit les commentaires de revue de code de 50 %.

8.4. SonarQube

- **Utilisation :**
 - Installer : `docker run -d -p 9000:9000 sonarqube.`
 - Scanner : `sonar-scanner -Dsonar.projectKey=projet -Dsonar.sources=..`
- **Exemple : Analyse bugs, dettes techniques.**

- **Intégration CI/CD :**

```
- name: Exécuter SonarQube
  run: sonar-scanner -Dsonar.projectKey=projet -Dsonar.sources=.
```

- **Impact :**

- Réduit les dettes techniques (ex. : 40 % de code smells en moins).

8.5. Dependabot

- **Utilisation :**
 - Activer sur GitHub : Paramètres > Sécurité > Activer Dependabot.
- **Exemple : Crée des PR pour requests avec CVE.**
- **Intégration CI/CD :** Auto-exécuté sur GitHub.
- **Impact :**
 - Élimine les vulnérabilités (ex. : 10 CVE corrigés en une semaine).

8.6. Bandit

- **Utilisation :**
 - Installer : `pip install bandit`.
 - Exécuter : `bandit -r projet/`.
- **Exemple :** Détecte `eval()` ou mots de passe codés en dur.
- **Intégration CI/CD :**

yaml

```
- name: Exécuter Bandit
  run: bandit -r . --exit-zero
```

- **Impact :**
 - Évite les failles comme l'injection SQL.

8.7. Swagger (drf-yasg)

- **Utilisation :**
 - Installer : `pip install drf-yasg`.
 - Accéder : `/swagger/`.
- **Configurer dans Django :**

python

```
INSTALLED_APPS = ['drf_yasg', ...]
```

- **Intégration CI/CD :**

yaml

```
- name: Générer docs Swagger
  run: python manage.py generate_swagger > swagger.json
```

- **Impact :**
 - Réduit le temps d'intégration API de 25 %.

8.8. Sphinx

- **Utilisation :**
 - Installer : `pip install sphinx`.
 - Initialiser : `sphinx-quickstart`.
 - Générer : `make html`.
- **Exemple :**

rst

```
.. function:: add(a, b)
   Additionne deux nombres.
   :param a: Premier nombre
   :param b: Deuxième nombre
   :return: Somme
```

- **Intégration CI/CD :**

yaml

```
- name: Générer docs Sphinx
  run: make html
```

- **Impact :**
 - Améliore la maintenabilité (ex. : 20 % d'efficacité en plus).

8.9. cProfile

- **Utilisation :**
 - Exécuter : `python -m cProfile -s time script.py`.
- **Exemple : Identifie fonctions lentes.**
- **Intégration CI/CD :**

yaml

```
- name: Exécuter cProfile
  run: python -m cProfile -o profile.out script.py
```

- **Impact :**
 - Optimise les performances (ex. : script 15 % plus rapide).

8.10. memory_profiler

- **Utilisation :**
 - Installer : `pip install memory-profiler`.
- **Exemple :**

python

```
from memory_profiler import profile
@profile
def fonction():
    return [i for i in range(1000000)]
```

- **Intégration CI/CD :**

yaml

```
- name: Exécuter memory_profiler
  run: python -m memory_profiler script.py
```

- **Impact :**
 - Réduit les fuites mémoire (ex. : 500 Mo économisés).

8.11. py-spy

- **Utilisation :**
 - Installer : pip install py-spy.
 - Exécuter : py-spy record -o profile.svg --pid <PID>.
- **Intégration :** Surveillance en production.
- **Impact :**
 - Identifie les goulots d'étranglement (ex. : 20 % CPU optimisé).

8.12. Pipeline CI/CD Exemple :

yaml

```
name: Pipeline Qualité Python
on: [push]
jobs:
  qualite:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Configurer Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.9'
      - name: Installer dépendances
        run: pip install pytest pytest-cov pylint flake8 bandit
      - name: Tests pytest
        run: pytest --cov=./ --cov-report=xml
      - name: Pylint
        run: pylint *.py --fail-under=8
      - name: flake8
        run: flake8 . --max-line-length=88
      - name: Bandit
        run: bandit -r . --exit-zero
```

Impact sur la Qualité

- **Fiabilité** : pytest/SonarQube réduisent les bugs (ex. : 95 % détectés).
- **Maintenabilité** : Pylint/flake8/Sphinx améliorent la lisibilité (ex. : 30 % moins de refactoring).
- **Sécurité** : Bandit/Dependabot éliminent les failles (ex. : 10 CVE corrigés).
- **Performance** : cProfile/memory_profiler/py-spy optimisent (ex. : 15 % plus rapide).